```
      <a href="{% url 'venues' %}?page={{page.next_page_number}}"> Next</a>
    {% endif %}
  {% endif %}
{% endblock content %}
```

E.4 Chapter 5

In chapter 5 you were introduced to the Django Admin, a built-in tool kit for managing all of your ORM models. The Django Admin is built using the same techniques you use in your own projects and is quite customizable.

Exercise 1

Add search capabilities to the `BandAdmin` class.

To add search to a Django `ModelAdmin` class, you add the `.search_fields` member to it, as shown in the following listing.

Listing E.18 A search field for the `BandAdmin` class

```
# RiffMates/bands/admin.py
...
@admin.register(Band)
class BandAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "show_members")
    search_fields = ("name",)
```

The Band class can now be searched based on its .name field.

Exercise 2

Add default sorting by name to the `Band`, `Venue`, and `Room` classes.

The `Meta` inner class is used to change the behavior of your model classes. You use the `ordering` member list to determine default sorting behavior. The following listing shows the `Meta` inner class on the `Band`, `Venue`, and `Room` classes.

Listing E.19 Adding default ordering to a model

```
# RiffMates/bands/models.py
...
class Band(models.Model):
    ...
    class Meta:
        ordering = [ "name", ]
    ...

class Venue(models.Model):
    ...
    class Meta:
```

The Meta inner class provides additional control information to the Django ORM.

The ordering member of Meta dictates the default sorting on a queried model; more than one field name can be provided.